Unoeste
Campus I Rua José Bongiovani, 700
Cidade Universitária CEP 19050 900
Fone (18) 3229 2003

Faculdade de Informática de Presidente Prudente
Fone (18) 3229 1060
www.unoeste.br/fipp
e-mail: fipp@fipp.unoeste.br

FIPP
FACULDADE DE INFORMÁTICA
DE PRESIDENTE PRUDENTE
CAMPUS I

Estruturas de Dados II – 2026 – Semestre 2

Professor: Francisco Assis da Silva

Exercício Prático em dupla: Árvore de Huffman para codificar **palavras** de uma frase

Você deverá fazer **2 programas separados** (para serem executados separadamente) em .cpp diferentes.

O primeiro program deverá ter:

a) Um algoritmo para construção da árvore de Huffman (utilize uma **lista encadeada ordenada** no processo de construção da árvore). A árvore não deverá ter a palavra e sim um símbolo (numérico, um código!). Os espaços em branco da frase também devem ser considerados e terá um símbolo na árvore. A frequência das palavras e dos espaços deverá ser contada e armazenada na lista do item b) antes de montar a árvore.

b) Uma lista (de registos) para armazenar: o símbolo, a palavra, a frequência e os códigos de Huffman. Os dados da lista, com exceção da frequência, deverão ser **gravados em arquivo** em disco (**arquivo binário**).

Exemplo de frase para usar na construção da árvore de huffman (ignore a pontuação e tamanho da caixa dos caracteres!). Utilize frases com palavras repetidas!!! Exemplo:

"Amar é sonhar, sonhar é viver, viver é curtir, curtir é amar. Cada um terá uma escolha, cada um fará a escolha, cada um escolhe a sua escolha. Levaremos um tempo para crescer, levaremos um tempo para amadurecer, levaremos um tempo para entender, levaremos um tempo para envelhecer, levaremos um tempo para morrer."

c) Após montar a lista (de registros) e a árvore, você deve exibir ambas na tela. No caso da árvore deve ser exibida no formato de árvore (vertical ou horizontal).

d) Codificação de uma frase e armazenamento do resultado (uma sequência de bits sem espaços, que deverá ser gravada **byte a byte**! Para montar um byte com 8 bits você pode utilizar **union**).
Obs: Utilize **uma frase diferente** da que foi usada na construção da árvore!

O segundo programa deverá ter:

a) Um algoritmo que abra os arquivos **binário** (registros) e o arquivo da **frase codificada**, faça a decodificação da frase e mostre na tela. Para fazer a decodificação monte uma árvore binária a partir dos códigos de huffman armazenados da tabela (**arquivo binário**), e com varreduras na árvore seguindo as sequências binárias você chegará nas folhas, onde estão os códigos das palavras. Vá no arquivo, busque o código e pegue a palavra ou o espaço em branco, concatene em uma string e no final você terá a frase decodificada. Após ter a frase decodificada mostre na tela.

OBS: Não use scanf() ou gets(). Faça as chamadas de funções passando coisas fixas, ou seja, basta rodar os dois programas que eles já apresentam os resultados!
As frases usadas para a montagem da árvore de huffman e para codificação devem ser diferentes! No seu trabalho use um texto bem grande (use um arquivo texto), maior do que o exemplo, para montar a árvore e uma frase contendo algumas das palavras desse texto para codificar!